

BMTech Platform

Marketing site · Admin dashboard · Automated lead-discovery & scoring engine

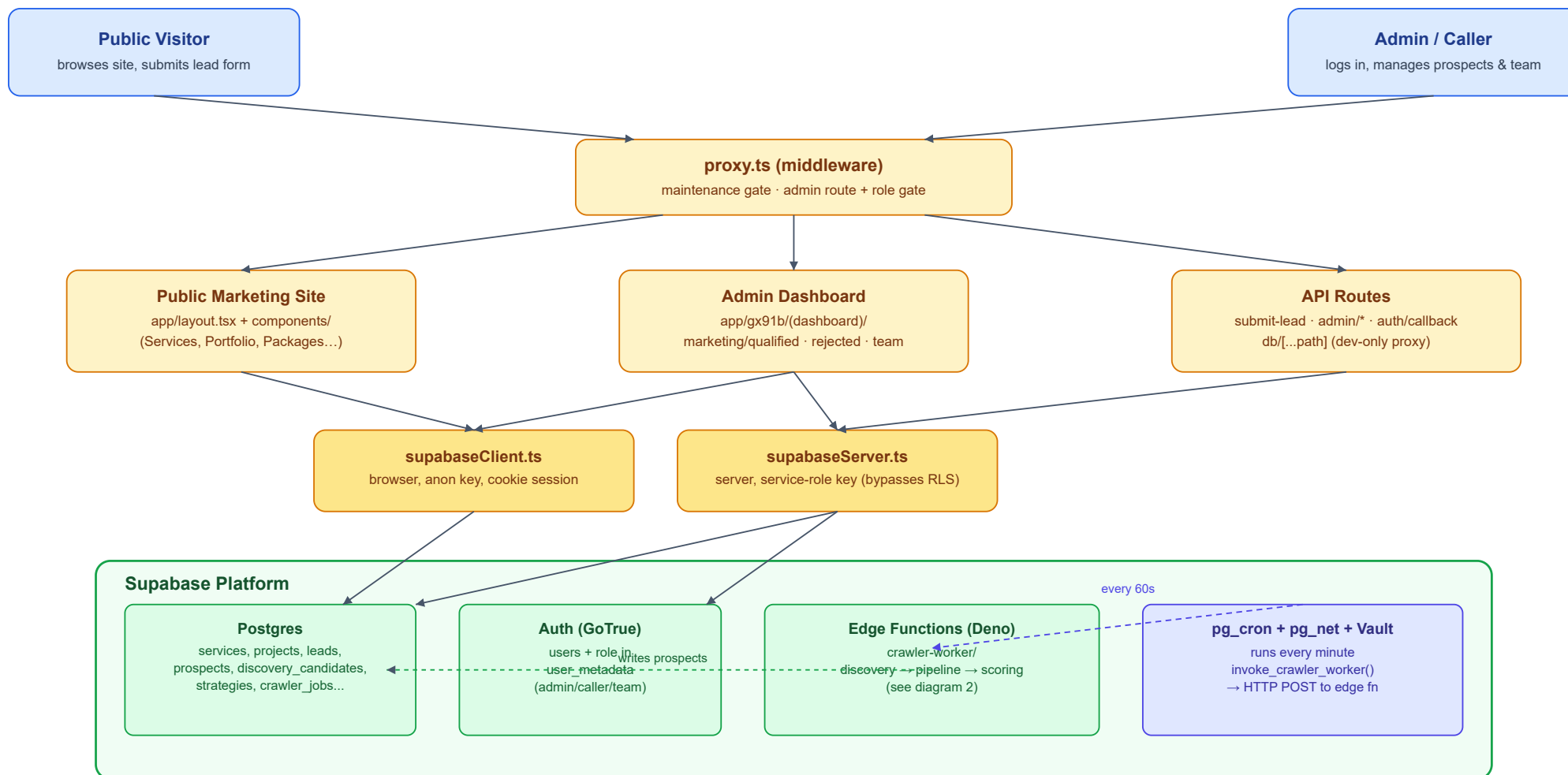
Prepared for onboarding / internal reference · Next.js 16 + Supabase (Postgres, Auth, Edge Functions)

Covers: request flow, data flow, the crawler/scoring pipeline, and where the real complexity lives.

1. System Overview

How a request moves from a browser, through the Next.js app, into Supabase — and how the background crawler fits in.

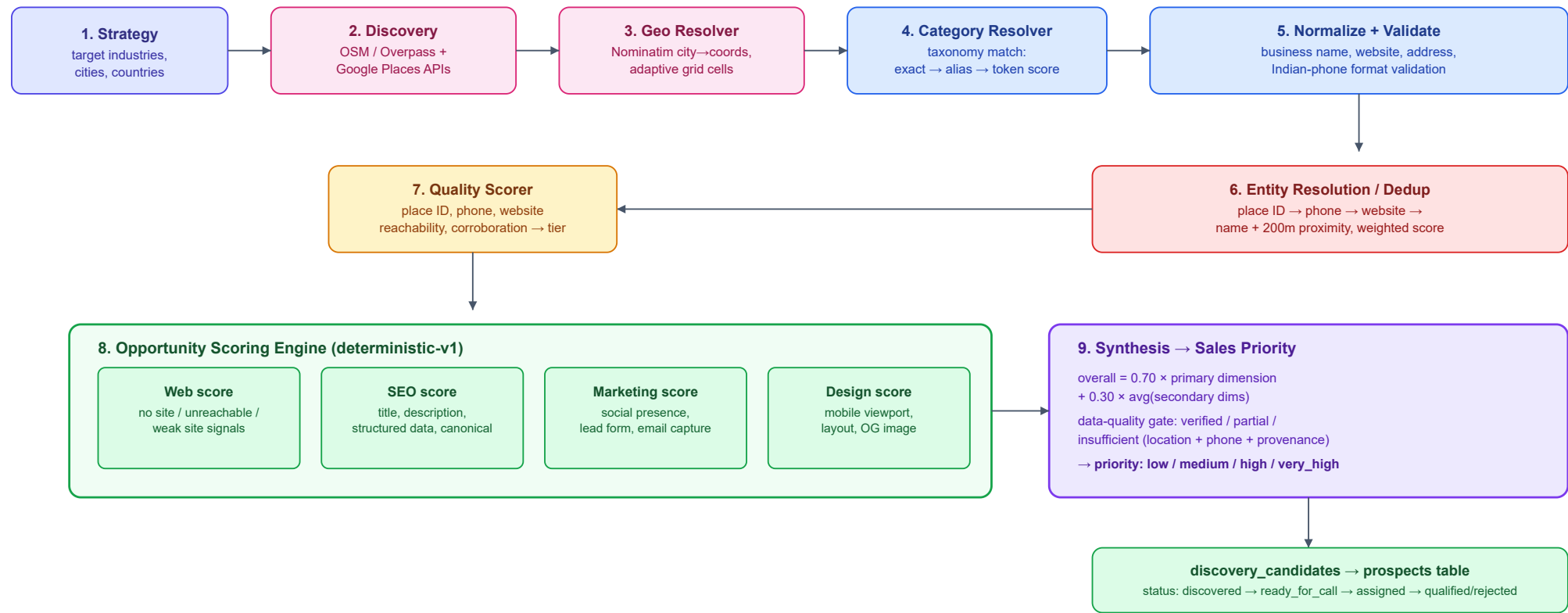
□ Client / Browser □ Next.js (Vercel) □ Supabase platform □ External APIs □ Scheduled / background



Why it feels tangled: there are three separate write paths into Postgres — the browser client (RLS-gated), the server client (service-role, bypasses RLS, used in API routes), and the edge function (its own service-role client in `_shared/supabase.ts`). Each has different trust assumptions, which is normal for this kind of app but easy to miss on first read.

2. Lead-Discovery & Scoring Pipeline

This is the most advanced part of the codebase — it runs unattended every minute inside `supabase/functions/crawler-worker/`.



Stage	File	What it actually does
Discovery	<code>discovery.ts</code>	Pulls raw candidates per industry taxonomy from OSM/Overpass and Google Places for the strategy's target area.
Geo resolve	<code>geo/resolver.ts</code> , <code>geo/grid.ts</code>	Turns a city name into a bounding box, then splits large areas into ~5.5km grid cells so discovery stays within provider limits.
Normalize	<code>pipeline/normalize.ts</code>	Strips legal suffixes, lower-cases and cleans names/websites/addresses so later matching is consistent.

Phone validation	pipeline/phoneValidator.ts	Pure heuristic parser for Indian numbers — no external API — flags toll-free, wrong-length, and invalid formats.
Entity resolution	pipeline/entityResolver.ts , pipeline/deduplicate.ts	Weighted match score (place ID 50, phone 40, website 30, name 10–20, proximity 5–15) decides SAME_ENTITY / LIKELY_SAME / DIFFERENT.
Quality scoring	pipeline/qualityScorer.ts	0–100 confidence score from source signals → tier (low/medium/good/high).
Opportunity scoring	scoring/config.ts , scoring/index.ts	Four independent dimension scores (web, SEO, marketing, design), each capped and weighted per SCORING_CONFIG_V1 .
Synthesis	lib/scoring (mirrored in edge fn)	Combines dimensions into one opportunity score and a sales-priority tier, gated by data-quality tier.

Notable duplication: the scoring types/config exist twice — once under [lib/scoring/](#) for the Next.js app and once under [supabase/functions/crawler-worker/scoring/](#) for the Deno edge function — because the two runtimes can't share a module easily. Keep both in sync when the formula changes.

3. Complexity Map & Where to Start Reading

Same codebase, ranked by how much context each part assumes.

Layer	Difficulty	Why
UI components	Easy	Standard React/Tailwind, one component = one concern (e.g. <code>PricingCard.tsx</code>).
Public data hooks	Easy	<code>useData.ts</code> is a thin fetch wrapper — same shape everywhere.
Auth & middleware	Medium	Cookie parsing, two auth flows (code exchange + implicit fallback), role gating in <code>proxy.ts</code> .
API routes	Medium	Mix of service-role and session-token calls; each route re-implements its own auth check.
Admin dashboard	Medium	Straightforward CRUD tables, but depends on knowing the <code>Prospect</code> status lifecycle.
Postgres schema + cron	Hard	Vault secrets, <code>SECURITY DEFINER</code> functions, <code>pg_net</code> HTTP calls from SQL.
Crawler pipeline	Hardest	Multi-stage pipeline, weighted scoring, duplicated logic across two runtimes.

If you want to understand...	Start here
How the public site gets its content	<code>hooks/useData.ts</code> → <code>lib/supabaseClient.ts</code>
How a lead form submission is stored	<code>app/api/submit-lead/route.ts</code>
How admin login/roles work	<code>proxy.ts</code> → <code>hooks/useAuth.ts</code>
How a prospect becomes "qualified"	<code>types/marketing.ts</code> (<code>ProspectStatus</code>) → dashboard pages
How new prospects appear automatically	<code>20260902_crawler_cron_heartbeat.sql</code> → <code>crawler-worker/index.ts</code>
How opportunity scores are calculated	<code>lib/scoring/config.ts</code> (read this before the pipeline code)

Practical tip: don't try to read this codebase top-to-bottom. Pick one *user journey* — e.g. "visitor submits a form" or "a prospect gets auto-discovered and scored" — and trace only the files on that path. Each journey only touches 4–6 files despite the repo looking huge.